A Deliberation Layer for a Meeting-Room Assistant Robot

William Janowsky
wilja141@student.liu.se
TDDE05 AI Robotics, Group 04, Linköping University

Abstract

This report builds a deliberation layer for a meeting-room robot, based on temporal-numeric PDDL2.1 planning solved by Aries in anytime mode. The layer schedules meetings against rooms and chair counts under capacity, attendance, and deadline constraints. Two reactive scheduling alternatives (arrival-order and earliest-deadline) are implemented in the same architectural slot as baselines. On a hand-built scenario the Aries-anytime plan reaches the analytical makespan lower bound, while the deadline-aware scheduler ends 1620 s later because a locally-cheap chair-source choice closes off later options it cannot see. The advantage generalises across 125 random instances: at instances with 7 to 11 meetings, Aries-anytime's median makespan is 16–24% shorter than deadline-aware, and arrival-order's success rate decays to 4% at 11 meetings. Dispatching both plans through Nav2 in simulation reveals that the planner's single chair-move duration assumption underestimates observed wall-clock on the measured Scenario 1 room pairs by roughly 2×, yet the deliberative advantage survives this modelling error because the structural lead is larger than the asymmetric exposure it suffers.

1 Introduction

Autonomous robots that must reason about future tasks before acting need a deliberation layer in their software, the architectural slot that handles long-horizon, future-aware planning in the hybrid pattern described in Murphy's textbook [1]. What occupies that slot shapes how the robot reasons about its tasks and what it commits to before acting.

This report concerns the deliberation layer of a meeting-room assistant robot. The robot is built on the Robot Operating System version 2 (ROS 2) [2] and uses Nav2 [3] as its navigation stack. Experiments are conducted in a TurtleBot-based 2D simulator that drives the same Nav2 stack against a static office map. The robot receives meeting requests with attendance counts, durations, and optional time windows, and places each meeting in a room with sufficient capacity and chairs, moving chairs between rooms as needed. The objective is to minimise the makespan: the latest meeting end-time across the plan. The problem is a variant of resource-constrained scheduling with deadlines, expressible as a temporal-numeric planning instance in the Planning Domain Definition Language version 2.1 (PDDL2.1) [4].

The deliberation layer built here solves the PDDL2.1 instance with Aries [5] in anytime mode, returning a globally-reasoned plan at the cost of search wall-clock. As baselines, two reactive scheduling alternatives are implemented in the same architectural slot: arrival-order (FIFO) and earliest-deadline. Each commits one meeting at a time without look-ahead, producing plans in microseconds but missing decisions whose cost only becomes visible later. All three consume the same inputs, so the comparison isolates what look-ahead in this slot is worth.

The hypothesis is that Aries-anytime's look-ahead pays off on instances with globally-coupled decisions, and that its plans execute correctly when dispatched on the simulator. Section 2 describes the architecture, formal problem, the two implementations, and the simulation execution path. Section 3 reports three experiments: a hand-built scenario where the deliberative implementation reaches an analytical bound the reactive schedulers cannot match, a plan-level distributional comparison across 125 random instances, and a sim-side execution comparison dispatching both plans through Nav2. Section 4 interprets the results.

2 Method

This section describes the deliberation layer of the meeting-room assistant robot: its place in the ROS 2 graph, the scheduling problem it solves, the PDDL2.1 encoding used to express that problem, the two implementations compared in this report (a deliberative one based on Aries in anytime mode and two reactive alternatives), and the simulation path through which a plan becomes robot behaviour.

2.1 Architecture

In Murphy's hybrid pattern [1] the deliberation layer plans over the future the reactive layers cannot see. Figure 1 places that slot in the meeting-room robot as a single ROS 2 [2] node, the mission controller. It receives meeting requests, queries the current chair-room distribution from the knowledge database (kDB) by SPARQL, and emits a Task Specification Tree (TST). The kDB itself is populated by a virtual environment sensor that commits observations from the robot's exploration sweep.

The executor refines each TST leaf into an action goal for one of three capability nodes (*explore*, *move chair*, *assign*), and each chair move ultimately becomes Nav2 [3] navigation goals against the robot's body. A periodic callback re-plans whenever the unassigned-meeting set changes and dispatches a new TST. Once a TST is dispatched it runs to completion, and the world over which deliberation runs is the robot's own belief and not a global ground truth.

2.2 Formal problem

Let R be the set of rooms, each with capacity $\mathrm{cap}(r)$ and initial chair count $n^0(r)$. Each meeting $m \in M$ carries an attendance $\mathrm{att}(m)$, a duration $d(m)$, and (optionally) an earliest start $\mathrm{es}(m)$ and latest end $\mathrm{le}(m)$. The deliberation layer picks for each meeting an assigned room $x_m \in R$ and start time t_m, and a trajectory ρ_c for each chair. Chair-move travel time is the constant $\delta = 60$ s. Writing $I(m) = [t_m, t_m + d(m))$ for the interval during which meeting m holds its room and J_i for the interval during which the i-th chair

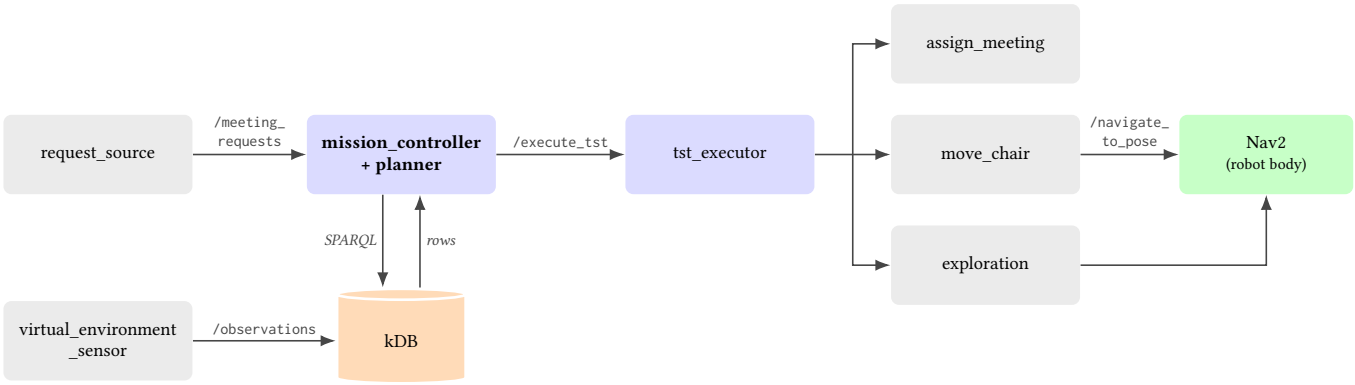


Figure 1: The deliberation layer in the ROS 2 graph. Blue boxes are nodes implemented in this work.

move holds the robot, a feasible plan must satisfy

$$\mathrm{cap}(x_m) \geq \mathrm{att}(m), \tag{1}$$

$$\left|\{c : \rho_c(t_m){=}x_m\}\right| \geq \mathrm{att}(m), \tag{2}$$

$$I(m_1) \cap I(m_2) = \emptyset \text{ if } x_{m_1}{=}x_{m_2}, \tag{3}$$

$$J_i \cap J_j = \emptyset \text{ for } i \neq j, \tag{4}$$

$$\mathrm{es}(m) \leq t_m,\ t_m + d(m) \leq \mathrm{le}(m) \text{ when set.} \tag{5}$$

In words: (1) the assigned room can seat the attendees, (2) the room has enough chairs when the meeting starts, (3) two meetings in the same room cannot overlap in time, (4) the single robot can only do one chair move at a time (so chair moves serialise), and (5) any time-window on a meeting is respected. Equation (4) ties the formulation to the physical hardware. The objective is to minimise the makespan $\max_m(t_m + d(m))$. The problem is a variant of the Resource-Constrained Project Scheduling Problem [6], expressed for a planner as a PDDL2.1 [4] instance with two durative actions (chair-move, meeting-assignment), extended with PDDL2.2's Timed Initial Literals [7] to encode the windows.

One encoding choice is worth flagging: chairs are represented as a per-room integer count chair_count(r), not as named PDDL objects. Naming each chair would introduce many equivalent states that differ only in the identities of otherwise interchangeable chairs, a known difficulty for state-based planners [8]. The count formulation removes this symmetry up front. Chair identity is recovered at execution, when the TST executor picks any chair currently in the source room to fetch. The chair-room counts the planner sees come from the kDB through SPARQL, populated by exploration, so the planner reasons over the robot's belief rather than a global ground truth.

2.3 Deliberative implementation

The chosen solver is Aries [5], invoked through the AIPlan4EU Unified Planning library [9]. Aries was the best fit for this domain among the Unified Planning temporal-numeric solvers tried on it (TAMER [10], for instance, did not return a plan within the operational budget on either benchmark scenario), and its decisive capability is an *anytime* mode that emits a stream of progressively improving feasible plans instead of a single one.

A single-pass call to Aries' `OneshotPlanner` returns the first feasible plan its search reaches, which on both benchmark scenarios is the same 6601 s plan, 1921 s and 301 s above the Scenario 1 and 2 optima. A single call is therefore not competitive. The anytime paradigm [11] instead collects a stream of progressively improving candidates and picks the best.

The planner keeps the candidate with the lowest lexicographic score $\mu(P) = (\mathrm{makespan}(P), \#\mathrm{moves}(P))$ seen so far, with the move-count tiebreak discouraging redundant chair flow. A short post-processing pass strips direct reverse-pair moves the search occasionally retains. Collection stops at a hard budget T or after s seconds without improvement, whichever comes first. Because the deliberation layer is offline, T and s are deployment knobs rather than real-time constraints. The values used here are $T = 60$ s and $s = 10$ s, sufficient for the experiments of Section 3.

2.4 Reactive alternatives

The alternative to deliberation is a scheduler with no look-ahead in the same architectural slot. Two are introduced: *arrival-order* (FIFO), which processes meetings in request order, and *deadline-aware*, which processes meetings in order of their latest-end constraint. Both follow the same outer loop:

(1) Sort the meetings (by arrival order for FIFO, or by latest end for deadline-aware).
(2) For the next meeting in order, find the room with the earliest feasible end-time given current room and chair state.
(3) Schedule any chair moves required to that room immediately ahead of the meeting, serialised over the single robot, and lock the source room for each move's duration.
(4) Commit the assignment, advance the state, and proceed to the next meeting.

Nothing in this loop depends on a future meeting, and that is what makes the procedure reactive. The pseudocode is given as Algorithm 1.

2.5 Simulation execution

Plans from either implementation are dispatched through the same execution path. The TST executor walks the tree in plan order,

Algorithm 1 Reactive scheduler (FIFO or deadline-aware).

Require: rooms, pending meetings, move duration δ
Ensure: a plan, or per-meeting infeasibility flag
sort meetings by arrival order *or* by earliest deadline
initialise per-room chair counts, room busy-intervals, robot-free time
for each meeting m in order **do**
  $best \leftarrow$ none
  for each room with capacity $\geq$ att(m) **do**
    compute chair shortfall and check donors
    find earliest room slot fitting moves + meeting, respecting m's window and the robot's free time
    if slot exists and ends earliest so far **then**
      $best \leftarrow$ (room, moves, meeting start)
    end if
  end for
  if *best* found **then**
    commit moves and assignment, advance state
  else
    mark m infeasible
  end if
end for

each chair-move leaf drives Nav2 source-to-destination and updates the kDB belief, and each meeting-assignment leaf publishes a `MeetingAssignment` message. The simulator does not enforce modelled meeting durations (an hour for Big Review is impractical). The standing assumption is that meetings take their modelled duration once they begin. The only observable variance is each chair-move's wall-clock duration (Nav2 round-trip time), and every other milestone is *derived* from the plan structure and those measured durations.

3 Results

Three experiments support the central claim: a case study where Aries-anytime reaches an analytical lower bound on a hand-built scenario (Section 3.1), a plan-level comparison of the three schedulers on the hand-built scenarios and on 125 random instances (Section 3.2), and a simulator dispatch of both complete Scenario 1 plans comparing predicted with observed milestones (Section 3.3). The hand-built scenarios share the meetings of Table 1. *Scenario 1* has rooms A(5, 2), B(3, 3), C(4, 4), D(5, 5), where each entry is (capacity, initial chairs). *Scenario 2* reduces Room C's capacity from 4 to 3, forcing Lunch out of C.

Table 1: Meetings used in both hand-built scenarios.

Meeting	Attendees	Duration (s)	es (s)	le (s)
Standup	3	900	–	900
Lunch	4	2700	–	–
Big Review	5	3600	–	3700
Workshop	5	3600	–	6000
Demo	3	1800	2700	–

3.1 Plan structure on Scenario 1

Scenario 1 is constructed to isolate a single coupled chair-source decision. It serves as an illustrative case study of the mechanism, with the random instances of Section 3.2 carrying the generalisation evidence. It admits an analytical makespan lower bound, derived from three forced room placements. Big Review's deadline (le = 3700 s) and 3600 s duration force it into Room D, the only cap-5 room with five chairs in place. Placing it in Room A would require at least three chair moves first and therefore end no earlier than 3780 s, past the deadline. Workshop cannot then follow Big Review in Room D within a 4680 s schedule, so it must run in Room A and receive three chairs. To preserve Room C for Lunch, these chairs must be supplied from Room B. Since Standup occupies Room B until $t = 900$ s, the three serial moves finish no earlier than $t = 1080$ s, and Workshop finishes no earlier than 4680 s.

Figure 2 shows the Aries-anytime plan, which respects every constraint of Section 2.2 and reaches the bound exactly with three chair moves.

3.2 Plan-level comparison

This experiment compares the deliberative Aries-anytime against the two reactive schedulers (FIFO and deadline-aware). FIFO is the no-information baseline whose collapse quantifies the floor any ordering heuristic must beat. The meaningful contest is deliberation against deadline-aware. Each scheduler is run to its stopping rule (Aries-anytime to the $T = 60$ s budget and $s = 10$ s stagnation cutoff of Section 2.3, the reactive schedulers to completion in sub-millisecond), so the comparison is about plan *output*, not about time-to-plan.

Table 2 reports each scheduler's output on the hand-built scenarios. On both, FIFO terminates with one meeting marked infeasible. The two failures originate in the same architectural property: a scheduler that commits to the first meeting in its queue without looking ahead cannot undo the resulting state. Concretely, FIFO assigns Lunch immediately, and that assignment then either deprives Workshop of an available chair source at $t = 0$ (Scenario 1, where Lunch occupies Room C) or blocks Big Review out of the only room that can meet its deadline (Scenario 2, where Lunch occupies Room D).

Table 2: Each scheduler on the hand-built scenarios. Dashes mark infeasible meetings.

	Scenario 1			Scenario 2		
	FIFO	Dead.	Aries	FIFO	Dead.	Aries
makespan (s)	–	6300	**4680**	–	6300	**6300**
chair moves	–	3	**3**	–	6	**3**
scheduled meetings	4/5	5/5	**5/5**	4/5	5/5	**5/5**

The deadline-aware scheduler avoids these failures by ordering on the latest-end constraint and so fits all five meetings on both scenarios. On Scenario 1 it processes Workshop ahead of Lunch and selects Room C as Workshop's chair donor. This forces Lunch into Room D after Big Review ends, and the plan finishes at 6300 s, which is 1620 s after Aries-anytime's analytical bound. On

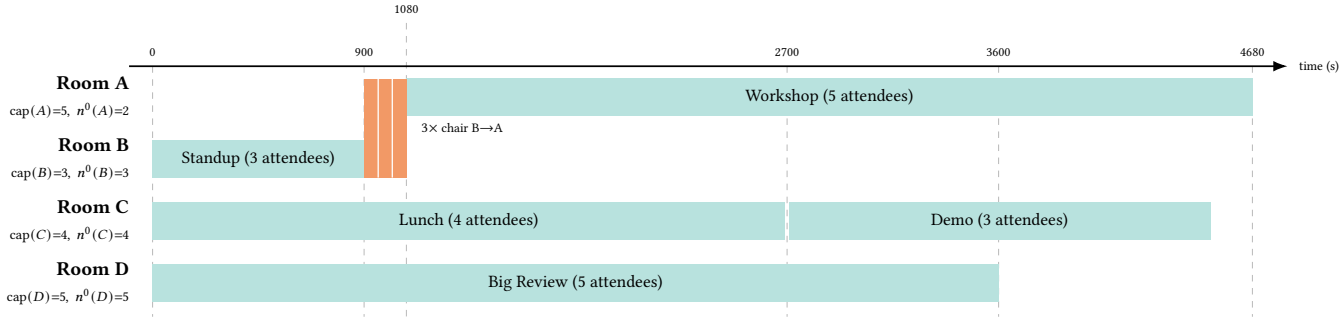


Figure 2: Aries-anytime plan on Scenario 1. Blue bars are meetings, the orange band is the three serial 60 s chair moves B→A.

Scenario 2 Room C's reduced capacity removes the coupled chair-source decision, and the two schedulers correctly tie on makespan, with a gap of three fewer chair moves (3 versus 6) for the deliberative implementation. The tie shows the deliberative advantage is specific to scenarios with a coupled decision, not a general claim that Aries always wins. The mechanism behind the Scenario 1 gap is the subject of Section 4.1.

To check that these observations generalise, each scheduler is then run on 125 random instances against the Scenario 1 building: 25 instances at each of $N = 3, 5, 7, 9, 11$, where N is the number of meetings drawn from a twelve-meeting pool into a single instance. Figures 3 and 4 report the per-scheduler success rate, median makespan, and median chair-move count at each N.

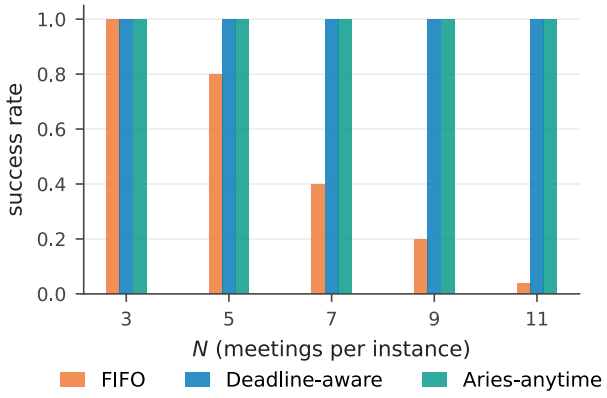


Figure 3: Success rate over 125 random instances, 25 per N.

All three schedulers handle every $N = 3$ instance, with identical median makespans of 3600 s. FIFO's success rate then decays at higher N (80%, 40%, 20%, 4% at $N = 5, 7, 9, 11$), while Aries-anytime and deadline-aware retain 100% success throughout. The two non-failing schedulers' median makespans are identical at $N = 3$ and 5 and diverge from $N = 7$ onwards. Deadline-aware's median runs 16%, 20%, and 24% longer than Aries-anytime's at $N = 7, 9, 11$ respectively, and the interquartile bands separate cleanly at high N. Chair-move counts move in the opposite direction at high N: Aries-anytime uses 5 moves to deadline-aware's 3 at $N = 11$, the lex-secondary criterion trading additional chair flow for shorter makespan.

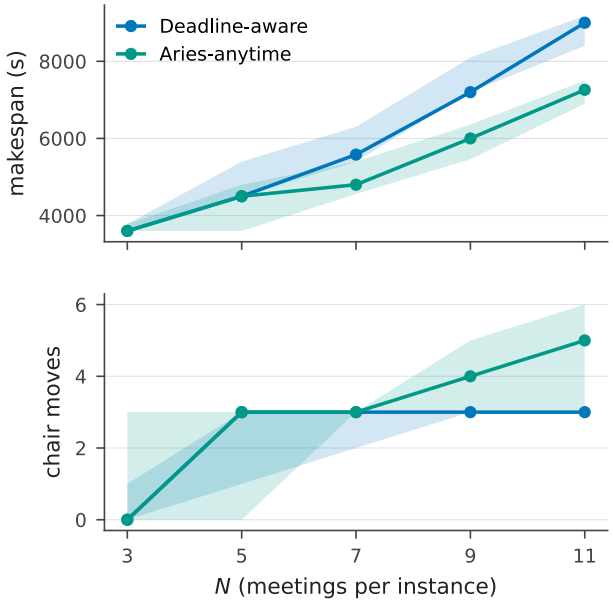


Figure 4: Aries-anytime versus deadline-aware: median makespan (top) and chair moves (bottom) per N, interquartile range bands.

3.3 Execution in simulation

The two complete Scenario 1 plans of Section 3.2 are each dispatched on the simulator 3 times through the same TST executor described in Section 2.5. A `result_logger` node captures per-`/move_chair` wall-clock between the action's `EXECUTING` and `SUCCEEDED` status transitions, and an observed makespan is derived for each trial from the plan structure under the standing assumption that meetings take their modelled duration once they begin.

Table 3 reports the headline numbers. The three Aries-anytime dispatches agree to within 5 s and the three deadline-aware dispatches agree exactly, confirming that observed makespan is determined by plan structure plus measured chair-move durations rather than by execution noise. The plan-level advantage of Section 3.2 survives execution: Aries-anytime's observed makespan is 4932 s (+5.4% above the predicted 4680 s) while deadline-aware lands exactly at its predicted 6300 s, narrowing the 1620 s plan-level gap to 1368 s observed.

The reason for the asymmetry is plan structure. Aries-anytime places its three chair moves on the binding critical path. Workshop cannot start until the B→A moves complete, so any per-move wall-clock above $\delta = 60$ s shifts Workshop's end. Deadline-aware places its chair moves off the critical path. Lunch in Room D is gated by Big Review's end at $t = 3600$ s, by which time the three C→A moves have long completed, so the same modelling gap is absorbed. The per-move duration also varies by room pair: the planner uses a single $\delta = 60$ s constant, but B↔A averages 144 s in execution while C↔A averages 82 s, reflecting Nav2 path-length differences invisible to the encoding.

The execution also exposes that the planner's single $\delta = 60$ s assumption underestimates observed Nav2 wall-clock on the two measured room pairs by roughly 2× (means 82 s and 144 s), yet the deliberative advantage survives this modelling error: the 1620 s structural lead is large enough that the asymmetric δ-exposure narrows but does not close it. This survival is scenario-dependent. A configuration where the deliberative plan's chair-moves were longer than the reactive plan's could narrow the gap further.

Table 3: Execution results on Scenario 1 over 3 trials per scheduler.

	Aries-anytime	Deadline-aware
predicted makespan (s)	4680	6300
observed makespan (s)	4932	6300
spread across 3 dispatches	5	0
mean chair-move duration (s)	144 (B↔A)	82 (C↔A)
chair moves on critical path	yes (Workshop)	no (Lunch in D)

4 Discussion

4.1 What the experiments show

The Scenario 1 result of Section 3.2 concentrates the report's central observation in a single decision. Both schedulers face the same problem: Workshop needs three chairs delivered to Room A. The deadline-aware scheduler, processing meetings in earliest-deadline order, picks Room C as the donor because it is the first room not currently busy at $t = 0$. This commits Lunch out of its only fitting room and pushes its end out by 1620 s. Aries-anytime instead defers Workshop's chair flow until Room B frees at $t = 900$, preserving Room C for Lunch. The two plans differ in one source-room decision, and that decision is worth 1620 s, over a third of Aries-anytime's 4680 s makespan. A reactive scheduler that commits this donor-room choice without look-ahead cannot recover the decision once Lunch is processed later. This is what Aries' search over the PDDL2.1 encoding buys.

FIFO's success rate (Figure 3) decays from 100% at $N = 3$ to 4% at $N = 11$, exactly the look-ahead failure that the chair-source argument predicts at scale: arrival-order commitments produce infeasibilities. Deadline-aware retains 100% success and is competitive at small N (tied medians at $N = 3$ and 5). Its makespan disadvantage (Figure 4) grows with N (16% at $N = 7$, 20% at $N = 9$, 24% at $N = 11$) because larger instances offer more opportunities for the chair-source-style coupled decisions that only deliberation can find. The random-instance results show that the advantage is not isolated to the hand-built Scenario 1 case. Where joint structure exists, deliberation pays off, and where it does not (small N), reactive is equally good and operationally instantaneous. These results motivate a hybrid architecture [1] that routes uniform-structure problems to a reactive scheduler and joint-structure problems to Aries-anytime. Evaluating such a routing strategy is a natural direction for future work. The cost is wall-clock that grows with the state space, but the deliberation layer is offline by construction and the 60 s budget used here is operationally cheap compared with the robot wall-clock it buys. Where this approach will eventually fail is at instance sizes where the state space outgrows any practical offline budget. The extensions that address this case are noted in Section 4.3.

The execution results in Section 3.3 show that the plan's structure also decides which modelling errors will reach the makespan: Aries puts chair flow on Workshop's critical path, where the single-δ assumption is exposed, while deadline-aware happens to keep it off.

4.2 Limitations

The deliberation layer is *offline* in the sense that a dispatched TST runs to completion without adaptation. Events during execution (a chair moved by a human, a meeting cancelled) do not interrupt the running plan, and a late request that arrives mid-execution is only addressed when the current TST finishes. The offline assumption is what allows $T = 60$ s and $s = 10$ s to be deployment knobs rather than real-time constraints, but it costs reactivity. The natural integration point for online deliberation is a plan-monitor loop in the style of PlanSys2 [12], a ROS 2 planning framework that integrates PDDL-based planning with execution monitoring. This would also require an executor that can safely interrupt an in-flight TST.

The objective is a single makespan criterion, and a real deployment would prefer a lexicographic order over (makespan, chair moves) reflecting energy and mechanical wear. The chair-move evidence cuts both ways. On Scenario 2 (Table 2) deadline-aware uses twice Aries-anytime's chair moves for the same makespan, while on random $N = 11$ instances (Figure 4) Aries-anytime spends 5 moves to deadline-aware's 3. Once two candidate plans differ on makespan, the move-count tiebreak cannot pull them back, and Aries spends chair moves freely to shave time. A two-stage solve (first the best makespan, then the fewest moves at that makespan) would address this, and the current one-pass tiebreak approximates it.

Evaluation runs on the laboratory's ROS 2 simulator. The simulator drives Nav2 end-to-end for each chair move and so exercises the part of the plan on which timing depends, but it abstracts away human corridor traffic and Nav2 path-planning failure modes such as recovery behaviours. Physical-hardware deployment would add modelling work that this evaluation did not have to do.

4.3 Future work and conclusion

Four concrete extensions follow from the limitations and observations above. Online re-planning, the missing reactive capability, is the most consequential and easiest to add: PlanSys2-style execution monitoring plus the existing TST executor would let the

deliberation layer re-plan on events. Multi-criterion optimisation generalises the move-count tiebreak into a principled secondary objective. Scaling deliberation to larger instances, such as buildings with tens of rooms and many concurrent meetings, needs domain knowledge injected into the planner: an admissible chair-move lower bound generalised from the Scenario 1 analytical bound tightens the search, and partitioning into room-disjoint sub-problems handles the case where decisions decouple.

The fourth extension is the one Section 3.3 most directly motivates: better chair-move duration modelling. The single $\delta = 60$ s constant underestimates Nav2 wall-clock on the measured Scenario 1 room pairs by roughly 2×, with comparable per-pair variation. A per-pair table (populated from a one-time exploration sweep) or a distance-based estimate could reduce much of the gap, at the cost of a larger encoding and potentially more expensive search. Online recalibration, which would adjust δ from observed durations during execution and re-plan when in-flight makespan diverges from prediction, ties this back to the online-deliberation point above.

The contribution is a deliberation layer for the meeting-room robot, formulated as temporal-numeric PDDL2.1 and implemented in Aries' anytime mode, shown to find plans that a competent reactive scheduler cannot when one decision affects another. The search cost is small compared to the robot wall-clock the layer's offline role can absorb. Execution in simulation confirms that the plan-level advantage propagates: the modelling gap between the planner's δ and observed chair-move wall-clock is large in absolute terms, but the Aries-anytime plan's structure means it does not close the gap to deadline-aware.

References

[1] Robin R. Murphy. *Introduction to AI Robotics*. MIT Press, 2 edition, 2019.

[2] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[3] Steven Macenski, Francisco Martin, Ruffin White, and Jonatan Ginés Clavero. The marathon 2: A navigation system. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2718–2725, 2020.

[4] Maria Fox and Derek Long. PDDL2.1: An extension to PDDL for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003.

[5] Arthur Bit-Monnot. Experimenting with lifted plan-space planning as scheduling: Aries in the 2023 IPC. In *Proceedings of the 11th International Planning Competition (IPC)*, 2023.

[6] Jacek Błażewicz, Jan Karel Lenstra, and Alexander H. G. Rinnooy Kan. Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics*, 5(1):11–24, 1983.

[7] Stefan Edelkamp and Jörg Hoffmann. PDDL2.2: The language for the classical part of the 4th international planning competition. Technical Report 195, Albert-Ludwigs-Universität Freiburg, Institut für Informatik, 2004.

[8] Nir Pochter, Aviv Zohar, and Jeffrey S. Rosenschein. Exploiting problem symmetries in state-based planners. In *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, pages 1004–1009, 2011.

[9] Andrea Micheli, Arthur Bit-Monnot, Gabriele Röger, Enrico Scala, Alessandro Valentini, Luca Framba, et al. Unified planning: Modeling, manipulating and solving AI planning problems in Python. *SoftwareX*, 29:102012, 2025.

[10] Alessandro Valentini, Andrea Micheli, and Alessandro Cimatti. Temporal planning with intermediate conditions and effects. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*, pages 9975–9982, 2020.

[11] Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996.

[12] Francisco Martín, Jonatan Ginés, Vicente Matellán, and Francisco J. Rodríguez. PlanSys2: A planning system framework for ROS2. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9742–9749, 2021.